Teaching a LEGO Robot to Solve a Rubik's Cube with Reinforcement Learning

The Idea

I have a LEGO Mindstorms robot on my desk. It's called MindCuber, and out of the box it can solve a Rubik's Cube using pre-computed lookup tables — essentially a giant cheat sheet baked into its tiny brain. It works, but there's no intelligence involved. The robot is just following instructions.

So I asked the obvious question: what if the robot had to *figure it out on its own?* No lookup tables, no human algorithms, no cheat sheets. Just a neural network, a GPU, and a lot of scrambled cubes.

This turned into a surprisingly deep project that touched reinforcement learning, neural network architecture, search algorithms, and robot control. It also involved a lot of failure, which — as with most projects — turned out to be the most educational part.

Why the Rubik's Cube is a Nightmare for AI

Before diving in, it's worth understanding why this is hard. The Rubik's Cube has 43 quintillion possible states. That's 43,252,003,274,489,856,000. There is exactly one solved state.

The fundamental problem is what RL researchers call "sparse reward." Imagine learning piano, but the only feedback you ever receive is either "that was the exact right song" or complete silence. No partial credit, no hints. That's what the AI faces — out of 43 quintillion states, only one gives a positive signal. Making random moves and hoping to stumble on the solution is statistically worse than winning the lottery.

It gets worse. A cube that's one move from solved and a cube that's twenty moves from solved can look almost identical. Counting correct stickers is deeply misleading — you can have 45 out of 54 stickers correct and still be 15 moves away. The AI can't trust simple measures of progress.

At each step there are 12 possible moves (6 faces × clockwise or counter-clockwise). After just 10 moves, there are roughly 62 billion possible sequences. After 20, that exceeds a sextillion. You can't brute force this.

The Hardware

The project runs on two machines:

NVIDIA DGX Spark — This is the brain. A compact edge AI workstation with a GB10 GPU and 128GB of unified memory. All neural network training and inference runs here. (Side muse… the PyTorch warning about CUDA capability 12.1 being unsupported has become a familiar friend at this point. It works anyway.)

LEGO SPIKE Hub — This is the body. A small MicroPython computer embedded in the robot, controlling 3 motors and 2 sensors. It can only turn the bottom face of the cube, so to execute a move on any other face, it has to physically tilt and rotate the entire cube to bring the target face to the bottom first. More on this later.

Attempt 1: Standard Reinforcement Learning (PPO)

The natural starting point. PPO (Proximal Policy Optimization) is one of the most popular RL algorithms — it's what OpenAI used to train their early robotics work. The idea is simple: the AI tries random moves, occasionally solves the cube by accident, and gradually learns which moves lead to rewards.

We added curriculum learning to help — start with 1-move scrambles, and only advance to harder ones after demonstrating competence. The setup:

```
class RubiksCubeEnv(gym.Env):
    # Observation: one-hot encoded cube state (324 floats)
    # Action: 12 discrete moves (F, F', B, B', L, L', R, R', U, U', D, D')
    # Reward: +100 for solving, -1 per move
```

Run 1 (394K parameter network, 2 hours training): Solved 1-6 move scrambles, then hit a wall at level 7. Zero percent on 10-move scrambles.

Run 2 (1.1M parameters, 7 hours training): Got slightly better at level 7, managed 9% on 10-move scrambles. But still fundamentally stuck.

This was the crucial moment. We'd tripled the network size and training time, used a better curriculum, and *still* hit the same wall. The problem wasn't tuning — it was fundamental to the approach. At scramble depth 7+, the probability of randomly stumbling onto the solution becomes vanishingly small. PPO simply cannot generate the training signal it needs.

Attempt 2: Autodidactic Iteration (ADI)

This is where things got interesting. ADI, introduced by McAleer et al. in the DeepCubeA paper (2019), takes a completely different approach. Instead of learning from rewards, it learns from *examples*.

The key insight is beautifully simple: we know what the solved state looks like. So we start there and scramble backwards. Every state we generate this way has a known distance from solved (the number of scrambles we applied) and a known best move (the reverse of the last scramble). This gives us millions of perfectly labelled training examples — for free.

```
def generate_training_data(num_samples, max_depth):
    for _ in range(num_samples):
        cube = RubiksCube()  # Start solved
        depth = random.randint(1, max_depth)
        for _ in range(depth):
            move = random.randint(0, 11)
            cube.move(move)
        # Now we know: this state is 'depth' moves from solved
        # and the best move is the reverse of the last scramble
```

The neural network has two heads: - **Value head**: "How far is this state from solved?" (regression) - **Policy head**: "What's the best move from here?" (classification)

Both are trained with standard supervised learning. No rewards, no exploration, no sparse signal problem. Just straightforward gradient descent on millions of labelled examples.

The Architecture

The v3 network uses residual connections — skip connections that let data bypass layers, making deeper networks trainable. This is the same idea from ResNet (He et al., 2015), arguably the most influential neural network paper of the last decade with over 200,000 citations.

```
class ResidualBlock(nn.Module):
    def forward(self, x):
        return self.dropout(self.relu(self.block(x) + x))
        #                                          ^^^ the skip connection
```

The full architecture: 324 inputs (one-hot encoded cube state) – input projection (4096 to 2048) – 4 residual blocks (2048 wide) – output projection (to 512) – dual heads. Total: 44.7 million parameters.

Beam Search

At solving time, the network doesn't just follow its top prediction greedily. It uses beam search — exploring multiple solution paths simultaneously and keeping the most promising ones. Think of it like navigating a maze with 1000 scouts: at each junction, you send all of them down different paths and keep the 1000 who seem closest to the exit.

The beam search is batched on the GPU for efficiency. All 1000 candidate states are evaluated in a single forward pass:

```
# Batched policy prediction — all beam states at once
batch_tensor = torch.FloatTensor(beam_states).to(device)
with torch.no_grad():
   policy_logits = model.policy_head(model._backbone_forward(batch_tensor))
    policy_probs = torch.softmax(policy_logits, dim=-1)
```

The Results

This is the bit I'm actually proud of.

ADI v1 (1M samples, 12.6M params, 20 minutes training): 100% on 10-move scrambles. Up from PPO's 9%. In twenty minutes.

ADI v2 (5M samples, same architecture, 3.5 hours): 25% on 20-move scrambles. Getting somewhere.

ADI v3 (20M samples, 44.7M params with residual connections, 42 hours): This is where beam width changed everything.

Scramble Depth	BW 100	BW 200	BW 500	BW 1000
5 moves	100%	100%	100%	100%
10 moves	100%	100%	100%	100%
15 moves	61%	80.5%	92.5%	99.0%
20 moves	30.5%	55%	82.5%	**93.5%**
25 moves	23%	43.5%	71%	87.0%

Without any retraining, just by searching more thoroughly at inference time, we went from 30.5% to 93.5% on 20-move scrambles. Solve time at beam width 1000 is about 4.3 seconds — perfectly acceptable for a robot that takes several seconds per physical move anyway.

One gotcha: the best checkpoint was at epoch 50 out of 100. The second half of training actually made things *worse* — classic overfitting. The loss kept decreasing but the solve rate degraded. If I hadn't been saving checkpoints every 5 epochs, I'd have ended up with a worse model after twice the training time. Lesson learned.

Connecting the Brain to the Body

This was the final piece — and arguably the most satisfying engineering challenge. The DGX Spark runs the AI solver; the SPIKE Hub controls the motors. They communicate over USB serial using a simple JSON protocol:

```
DGX --> SPIKE:  {"cmd": "scan"}
SPIKE --> DGX:  {"status": "ok", "data": [[0,1,3,...], [1,1,2,...], ...]}

DGX --> SPIKE:  {"cmd": "turn_face", "rot": 1}
SPIKE --> DGX:  {"status": "ok"}
```

The interesting problem is move translation. The AI thinks in abstract moves: F, R', U, B, etc. But the robot can only physically turn the bottom face. To execute, say, an R move, the robot has to:

1. Rotate the turntable to bring the right face to the front
2. Tilt the cube forward so the front face falls to the bottom
3. Now turn the bottom face (which is actually R)

This requires tracking the cube's physical orientation at all times. I built an orientation tracker that maintains a mapping between the AI's abstract face labels and the robot's physical positions:

```
class CubeOrientation:
    def tilt_forward(self):
        """front -> bottom, bottom -> back, back -> top, top -> front"""
        b, t, f, bk, l, r = self.faces
        self.faces = [f, bk, t, b, l, r]

    def get_sequence_to_bottom(self, target_face):
        pos = self.find_face(target_face)
        if pos == 'bottom':   return []
        elif pos == 'front':  return ['tilt_forward']
        elif pos == 'back':   return ['tilt_backward']
        elif pos == 'top':    return ['tilt_forward', 'tilt_forward']
        elif pos == 'left':   return ['rotate_cw', 'tilt_forward']
        elif pos == 'right':  return ['rotate_ccw', 'tilt_forward']
```

Getting this right required careful unit testing — 16 tests covering every face, every tilt direction, every rotation. The kind of code where an off-by-one error means the robot confidently executes the wrong move and makes the cube worse.

End-to-End Simulation

Before touching real hardware, I ran the full pipeline in simulation: scramble a virtual cube, scan it, solve it with the real ADI model, translate the moves through the orientation tracker, execute them on a simulated robot, and verify the cube ends up solved.

```
=== Trial 1/5 (20-move scramble, BW=1000) ===
Scramble: U B' R' F D L B L' B F' U L D' F D' U' R B D' L
Solution found: 26 moves in 4.44s
Result: SOLVED

=== Trial 2/5 ===  Result: SOLVED (24 moves)
=== Trial 3/5 ===  Result: SOLVED (30 moves)
=== Trial 4/5 ===  Result: SOLVED (24 moves)
=== Trial 5/5 ===  Result: SOLVED (24 moves)

5/5 solved and verified correct
```

All five 20-move scrambles solved and verified. The orientation tracking, move translation, and beam search all working together correctly.

What I Learned

Start simple and let failures guide you. We didn't jump straight to ADI. We tried PPO first, learned why it failed, improved the curriculum, learned why *that* still failed, and only then found ADI. Each failure taught us something essential about the problem. If we'd started with ADI, we wouldn't have understood *why* it works.

Inference-time compute is underrated. The single most impactful result was increasing beam width from 100 to 1000 — no retraining, just searching more thoroughly. This is the same principle behind recent advances in LLM reasoning (think chain-of-thought, tree-of-thought). Training gives you the knowledge; search gives you the performance.

Overfitting is sneaky. Our loss curve looked perfect — smoothly decreasing for 100 epochs. But the actual solve rate peaked at epoch 50 and declined after that. If you're only watching the loss, you'll miss this entirely. Always evaluate on the metric you actually care about.

The sim-to-real gap is real but manageable. The robot's colour sensor doesn't see perfect RGB values. Lighting conditions matter. Motor positions drift slightly. But the core architecture — AI on the DGX, motor control on the SPIKE Hub, JSON commands over USB — is robust enough to handle these imperfections.

The Numbers

What	Value
Network parameters	44.7 million
Training samples	20 million
Training time (v3)	42 hours
Best checkpoint	Epoch 50 / 100

What	Value
Solve rate (20-move, BW1000)	93.5%
Solve time (BW1000)	~4.3 seconds
Hardware (training)	NVIDIA DGX Spark, GB10 GPU
Hardware (robot)	LEGO SPIKE Hub, 3 motors, 2 sensors
Lines of Python	~2,500 across 6 files

What's Next

Two things on the list. First, data augmentation using the cube's 48-fold symmetry group — every training example can be rotated into 47 additional valid examples for free, effectively multiplying the dataset by 48×. Second, iterative label refinement, where the network's own value predictions replace the crude scramble-depth labels, potentially improving training signal quality.

And of course, the physical robot needs more testing. The simulation works perfectly; the real world, as always, will have opinions about that.

The code is on the DGX Spark at `~/Projects/mindcuber-rl/` if anyone wants to have a look. The model checkpoint is about 180MB and solves most cubes in under 5 seconds. Not bad for something that taught itself.